1.a.1)

```
# Import PyTorch
import torch

def create_tensor(shape, dtype=torch.float32):
    # Create tensor with given shape and datatype
    tensor = torch.zeros(shape, dtype=dtype)
    print(f"Tensor with shape {shape} and datatype {dtype} created:")
    print(tensor)

# Example: Create a 2x3 tensor of float32
demo_shape = (2, 3)
demo_dtype = torch.float32

create_tensor(demo_shape, demo_dtype)

# You can try other shapes and datatypes
# create_tensor((4, 5), torch.int32)
# create_tensor((3, 3), torch.float64)
```

1.a.2

```
# Import PyTorch
import torch


def create_tensor(shape, dtype=torch.float32):
    # Create tensor with given shape and datatype
    tensor = torch.zeros(shape, dtype=dtype)
    print(f"Tensor with shape {shape} and datatype {dtype} created:")
    print(tensor)
    return tensor


def tensor_operations():
    # Create two example tensors
    tensor1 = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
    tensor2 = torch.tensor([[5, 6], [7, 8]], dtype=torch.float32)


    print("Tensor 1:")
    print(tensor1)
    print("\nTensor 2:")
    print(tensor2)
```

```
    # Perform addition
    addition_result = torch.add(tensor1, tensor2)
    print("\nAddition Result:")
    print(addition_result)

    # Perform subtraction
    subtraction_result = torch.sub(tensor1, tensor2)
    print("\nSubtraction Result:")
    print(subtraction_result)

    # Perform multiplication (element-wise)
    multiplication_result = torch.mul(tensor1, tensor2)
    print("\nMultiplication Result:")
    print(multiplication_result)

tensor_operations()
= torch.mul(tensor1, tensor2)
print("\nMultiplication Result:")
print(multiplication_result)
```

1.a.3

```
# Import PyTorch
import torch


def create_tensor(shape, dtype=torch.float32):
    tensor = torch.arange(shape[0] * shape[1], dtype=dtype).reshape(shape)
    print(f"Original Tensor with shape {shape} and datatype {dtype}:")
    print(tensor)
    return tensor


def perform_operations(tensor):
    # Reshape the tensor
    reshaped_tensor = tensor.reshape(-1, 2)
    print("\nReshaped Tensor (-1, 2):")
    print(reshaped_tensor)
```

```
    # Slice the tensor (Extract 1st row and 1st 2 columns)
    sliced_tensor = tensor[0, :2]
    print("\nSliced Tensor (1st row, 1st 2 columns):")
    print(sliced_tensor)

    # Extract specific elements (e.g., element at (1,1))
    specific_element = tensor[1, 1]
    print("\nSpecific Element at (1,1):")
    print(specific_element)

# Example: Create a 2x3 tensor of float32
demo_shape = (2, 3)
demo_dtype = torch.float32
tensor = create_tensor(demo_shape, demo_dtype)

# Perform reshape, slice, and extract operations
perform_operations(tensor)
```

1.a.4

```
# Import TensorFlow
import tensorflow as tf

# Perform Matrix Multiplication
def matrix_multiplication(A, B):
    result = tf.linalg.matmul(A, B)
    print("Matrix Multiplication Result:")
    print(result.numpy())

# Find Eigenvalues and Eigenvectors
def find_eigen(A):
    eigenvalues, eigenvectors = tf.linalg.eigh(A)
    print("Eigenvalues:")
    print(eigenvalues.numpy())
```

```
    print("Eigenvectors:")
    print(eigenvectors.numpy())

# Example Matrices (2x2)
A = tf.constant([[4.0, -2.0], [1.0, 1.0]], dtype=tf.float32)
B = tf.constant([[1.0, 2.0], [3.0, 4.0]], dtype=tf.float32)

# Perform Operations
matrix_multiplication(A, B)
find_eigen(A)
```

1.b

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# XOR Input and Output Data
x_train = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

```
y_train = np.array([[0], [1], [1], [0]])

# Define a simple Neural Network model
model = keras.Sequential([
    layers.Dense(4, activation='relu', input_shape=(2,)),
    layers.Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=500, verbose=0)

# Evaluate the model
loss, accuracy = model.evaluate(x_train, y_train)
print(f"Training Accuracy: {accuracy * 100:.2f}%")

# Predict the outputs for XOR inputs
predictions = model.predict(x_train)
print("\nPredictions:")
for i, pred in enumerate(predictions):
    print(f"Input: {x_train[i]} => Predicted: {round(pred[0])}, Actual: {y_train[i][0]}")
```

3.a

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Generate Sample Data (Binary Classification)
np.random.seed(42)
x_train = np.random.rand(1000, 2)
y_train = (x_train[:, 0] + x_train[:, 1] > 1).astype(int)

x_test = np.random.rand(200, 2)
y_test = (x_test[:, 0] + x_test[:, 1] > 1).astype(int)

# Define a Simple Deep Neural Network Model
model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(2,)),
    layers.Dense(8, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Compile the Model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the Model
model.fit(x_train, y_train, epochs=20, batch_size=32, validation_split=0.2)

# Evaluate on Test Data
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")

# Predict on Test Data
predictions = (model.predict(x_test) > 0.5).astype(int)
print("\nSample Predictions:")
print(predictions[:10].flatten())
```

3.b

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# Load Iris dataset (a common multiclass dataset with 3 classes)
iris = datasets.load_iris()
X = iris.data
y = iris.target.reshape(-1, 1)

# One-hot encode the labels
encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y)

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2,
random_state=42)

# Build a simple deep forward network model
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(4,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(3, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=50, batch_size=16, verbose=1)

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

```
# Predict the class for test data
predictions = model.predict(X_test)
predicted_classes = np.argmax(predictions, axis=1)
actual_classes = np.argmax(y_test, axis=1)

print("\nPredicted Classes:", predicted_classes)
print("Actual Classes:", actual_classes)
```

In the Iris dataset, the:

4 Features are:

1. *Sepal Length*: the length of the sepal (a part of the flower)

2. *Sepal Width*: the width of the sepal

3. *Petal Length*: the length of the petal

4. *Petal Width*: the width of the petal

3 Classes are:

1. *Setosa*: a type of Iris flower

2. *Versicolor*: another type of Iris flower

3. *Virginica*: the third type of Iris flower

The goal is to predict which class (Setosa, Versicolor, or Virginica) a flower belongs to based on its 4 features (sepal length, sepal width, petal length, and petal width).

2.1

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Generate sample data
np.random.seed(42)
X_train = np.linspace(0, 10, 100).reshape(-1, 1).astype(np.float32)
y_train = 2.5 * X_train + np.random.normal(0, 2, (100, 1)).astype(np.float32)  # y = 2.5x +
noise

# Define model parameters
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))

# Define linear regression model
def linear_regression(X):
    return tf.matmul(X, W) + b

# Define loss function (Mean Squared Error)
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))

# Define optimizer
```

```
optimizer = tf.optimizers.SGD(learning_rate=0.01)

# Training function
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
    gradients = tape.gradient(loss, [W, b])
    optimizer.apply_gradients(zip(gradients, [W, b]))
    return loss

# Training loop
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(X_train, y_train)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f"Epoch {epoch}, Loss: {loss.numpy():.4f}")

# Plot results
plt.plot(range(epochs), loss_history, label='Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training Loss')
plt.legend()
plt.show()

# Plot the fitted line
plt.scatter(X_train, y_train, label='Data')
plt.plot(X_train, linear_regression(X_train).numpy(), color='red', label='Fitted Line')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Linear Regression Fit')
plt.legend()
plt.show()

print(f"Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}")
```

2.2

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Generate a toy dataset: House size (sq ft) vs Price ($1000s)
np.random.seed(42)
house_size = np.linspace(500, 4000, 100).reshape(-1, 1).astype(np.float32)  # House size in
sq ft
```

```
house_price = 50 + 0.1 * house_size + np.random.normal(0, 10, (100, 1)).astype(np.float32)
# Price in $1000s

# Normalize data
house_size_norm = (house_size - np.mean(house_size)) / np.std(house_size)
house_price_norm = (house_price - np.mean(house_price)) / np.std(house_price)

# Define model parameters
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))

# Define linear regression model
def linear_regression(X):
    return tf.matmul(X, W) + b

# Define loss function (Mean Squared Error)
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))

# Define optimizer
optimizer = tf.optimizers.SGD(learning_rate=0.01)

# Training function
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
    gradients = tape.gradient(loss, [W, b])
    optimizer.apply_gradients(zip(gradients, [W, b]))
    return loss

# Training loop
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(house_size_norm, house_price_norm)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f"Epoch {epoch}, Loss: {loss.numpy():.4f}")

# Plot results
plt.plot(range(epochs), loss_history, label='Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
```

```
plt.title('Training Loss')
plt.legend()
plt.show()

# Plot the fitted line
plt.scatter(house_size_norm, house_price_norm, label='Data')
plt.plot(house_size_norm, linear_regression(house_size_norm).numpy(), color='red',
label='Fitted Line')
plt.xlabel('Normalized House Size')
plt.ylabel('Normalized Price')
plt.title('Linear Regression Fit')
plt.legend()
plt.show()

print(f"Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}")
```

2.3

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Generate a toy dataset: House size (sq ft) vs Price ($1000s)
np.random.seed(42)
house_size = np.linspace(500, 4000, 100).reshape(-1, 1).astype(np.float32)  # House size in
sq ft
house_price = 50 + 0.1 * house_size + np.random.normal(0, 10, (100, 1)).astype(np.float32)
# Price in $1000s

# Normalize data
house_size_norm = (house_size - np.mean(house_size)) / np.std(house_size)
house_price_norm = (house_price - np.mean(house_price)) / np.std(house_price)

# Define model parameters
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))

# Define linear regression model
def linear_regression(X):
    return tf.matmul(X, W) + b

# Define loss function (Mean Squared Error)
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))

# Define optimizer
optimizer = tf.optimizers.SGD(learning_rate=0.01)

# Training function
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
    gradients = tape.gradient(loss, [W, b])
    optimizer.apply_gradients(zip(gradients, [W, b]))
```

```
    return loss

# Training loop
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(house_size_norm, house_price_norm)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f"Epoch {epoch}, Loss: {loss.numpy():.4f}")

# Plot Loss Function
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(epochs), loss_history, label='Loss', color='blue')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training Loss')
plt.legend()

# Plot the Learned Linear Relationship
plt.subplot(1, 2, 2)
plt.scatter(house_size, house_price, label='Data', color='gray')
plt.plot(house_size, (linear_regression(house_size_norm).numpy() * np.std(house_price)) +
np.mean(house_price), color='red', label='Fitted Line')
plt.xlabel('House Size (sq ft)')
plt.ylabel('House Price ($1000s)')
plt.title('Learned Linear Relationship')
plt.legend()

plt.show()

print(f"Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}")
```

2.4

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Generate a toy dataset: House size (sq ft) vs Price ($1000s)
np.random.seed(42)
house_size = np.linspace(500, 4000, 100).reshape(-1, 1).astype(np.float32)  # House size in
sq ft
house_price = 50 + 0.1 * house_size + np.random.normal(0, 10, (100, 1)).astype(np.float32)
# Price in $1000s

# Normalize data
house_size_mean = np.mean(house_size)
house_size_std = np.std(house_size)
house_price_mean = np.mean(house_price)
house_price_std = np.std(house_price)

house_size_norm = (house_size - house_size_mean) / house_size_std
house_price_norm = (house_price - house_price_mean) / house_price_std

# Define model parameters
```

```
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))

# Define linear regression model
def linear_regression(X):
    return tf.matmul(X, W) + b

# Define loss function (Mean Squared Error)
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))

# Define optimizer
optimizer = tf.optimizers.SGD(learning_rate=0.01)

# Training function
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
    gradients = tape.gradient(loss, [W, b])
    optimizer.apply_gradients(zip(gradients, [W, b]))
    return loss

# Training loop
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(house_size_norm, house_price_norm)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f"Epoch {epoch}, Loss: {loss.numpy():.4f}")

# Plot Loss Function
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(epochs), loss_history, label='Loss', color='blue')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training Loss')
plt.legend()

# Plot the Learned Linear Relationship
plt.subplot(1, 2, 2)
plt.scatter(house_size, house_price, label='Data', color='gray')
```

```
plt.plot(house_size, (linear_regression(house_size_norm).numpy() * house_price_std) +
house_price_mean, color='red', label='Fitted Line')
plt.xlabel('House Size (sq ft)')
plt.ylabel('House Price ($1000s)')
plt.title('Learned Linear Relationship')
plt.legend()

plt.show()

print(f"Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}")

# Predict new data points
new_house_sizes = np.array([[750], [1500], [3000], [4500]], dtype=np.float32)  # Example
new house sizes
new_house_sizes_norm = (new_house_sizes - house_size_mean) / house_size_std  #
Normalize new data
predicted_prices_norm = linear_regression(new_house_sizes_norm).numpy()  # Predict
predicted_prices = (predicted_prices_norm * house_price_std) + house_price_mean  #
De-normalize

# Print predictions
for size, price in zip(new_house_sizes.flatten(), predicted_prices.flatten()):
    print(f"Predicted price for house size {size} sq ft: ${price * 1000:.2f}")
```

```
# Install required library
!pip install tensorflow tensorflow-datasets -q

# Import libraries
import tensorflow as tf
import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt

# Load EMNIST dataset (Letters)
(ds_train, ds_test), ds_info = tfds.load(
    'emnist/letters',
    split=['train', 'test'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True
)
```

```
# Preprocessing function
def preprocess(image, label):
    image = tf.cast(image, tf.float32) / 255.0  # Normalize
    image = tf.transpose(image)  # EMNIST images are rotated, fix it
    image = tf.image.flip_left_right(image)
    image = tf.reshape(image, (28, 28))  # For RNN input
    label = label - 1  # Labels are from 1-26, shift to 0-25
    return image, label

# Apply preprocessing
ds_train = ds_train.map(preprocess).batch(128).prefetch(tf.data.AUTOTUNE)
ds_test = ds_test.map(preprocess).batch(128).prefetch(tf.data.AUTOTUNE)

# Build the RNN model
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(128, input_shape=(28, 28), return_sequences=False),
    tf.keras.layers.Dense(26, activation='softmax')  # 26 classes for A-Z
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(ds_train, epochs=5, validation_data=ds_test)

# Test the model
test_images, test_labels = next(iter(ds_test))
predictions = model.predict(test_images)

# Display a few predictions
for i in range(5):
    plt.imshow(test_images[i], cmap='gray')
    plt.title(f"True: {chr(test_labels[i].numpy() + 65)} | Pred: {chr(np.argmax(predictions[i]) +
65)}")
    plt.axis('off')
    plt.show()
```

🎯 Goal: Recognize handwritten characters (A-Z) using RNN in Google Colab.

🔢 1. Install Required Libraries

python
CopyEdit
```
!pip install tensorflow tensorflow-datasets -q
```

👉 This installs **TensorFlow** and a helper library to load ready-made datasets easily.

🧠 2. Import Libraries

python
CopyEdit
```
import tensorflow as tf
import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt
```

👉 These are the tools we'll use:

- tensorflow for the model
- tensorflow_datasets for loading the EMNIST data
- numpy for numbers
- matplotlib for showing images

📦 3. Load the Dataset

python
CopyEdit
```
(ds_train, ds_test), ds_info = tfds.load(
    'emnist/letters',
    split=['train', 'test'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True
)
```

👉 We’re using the **EMNIST Letters dataset** — it has handwritten **capital letters A-Z**.

- ds_train: training data

- ds_test: test data
- Each image is 28x28 pixels (black and white)

🛠️ 4. Preprocess Images

python
CopyEdit
```
def preprocess(image, label):
    image = tf.cast(image, tf.float32) / 255.0
    image = tf.transpose(image)
    image = tf.image.flip_left_right(image)
    image = tf.reshape(image, (28, 28))
    label = label - 1
    return image, label
```

👉 What this function does:

- Makes pixel values between 0 and 1 (/ 255)
- Rotates the image correctly (transpose + flip)
- Reshapes image so the RNN can read one row at a time
- Adjusts label from 1-26 → 0-25 (A = 0, Z = 25)

📤 5. Prepare the Dataset

python
CopyEdit
```
ds_train = ds_train.map(preprocess).batch(128).prefetch(tf.data.AUTOTUNE)
ds_test = ds_test.map(preprocess).batch(128).prefetch(tf.data.AUTOTUNE)
```

👉 We:

- Apply the preprocessing function
- Group data into batches of 128
- Use prefetch to make training faster

🧱 6. Build the RNN Model

python
CopyEdit
```
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(128, input_shape=(28, 28), return_sequences=False),
    tf.keras.layers.Dense(26, activation='softmax')
])
```

👉 What's inside the model:

- SimpleRNN(128): A basic RNN layer that reads 28 rows one-by-one.
- Dense(26): Final layer with 26 outputs — one for each letter A-Z.
- softmax: Makes the output a probability.

⚙️ 7. Compile the Model

python
CopyEdit
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

👉 We tell the model:

- Use adam optimizer to improve itself
- Use a loss function made for classification
- Show accuracy as performance

🚀 8. Train the Model

python
CopyEdit
```
model.fit(ds_train, epochs=5, validation_data=ds_test)
```

👉 Trains for 5 rounds (epochs) and checks accuracy using test data too.

🔍 9. Predict and Show Results

python
CopyEdit
```
test_images, test_labels = next(iter(ds_test))
predictions = model.predict(test_images)

for i in range(5):
    plt.imshow(test_images[i], cmap='gray')
    plt.title(f"True: {chr(test_labels[i].numpy() + 65)} | Pred: {chr(np.argmax(predictions[i]) + 65)}")
    plt.axis('off')
    plt.show()
```

👉 This:

- Picks some images from test set
- Shows the actual letter and what the model predicted

chr(... + 65) converts number (like 0–25) into capital letters A-Z.

Step	What's Happening
Load Data	Gets handwritten letters (A–Z)
Preprocess	Fix image orientation, normalize
Model	Simple RNN + Softmax
Train	Learns from data
Predict	Checks if model is guessing letters correctly

9. Demonstrate recurrent neural network that learns to perform sequence analysis for stock OC2,0C3,0C4 price.(google stock price)

Step 1: Install and import libraries

```
!pip install yfinance --quiet

import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Step 2: Download Google (GOOG) stock data
df = yf.download('GOOG', start='2020-01-01', end='2024-12-31')
df = df[['Open', 'Close', 'High', 'Low']]  # Focus on OC2,0C3,0C4

# Step 3: Normalize data
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)

# Step 4: Create sequences for training
def create_sequences(data, seq_length=60):
    x, y = [], []
    for i in range(len(data) - seq_length):
        x.append(data[i:i+seq_length])
        y.append(data[i+seq_length])  # predict next time step
    return np.array(x), np.array(y)

sequence_length = 60
X, y = create_sequences(scaled_data, sequence_length)

# Step 5: Split into train/test sets
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Step 6: Build RNN model with LSTM
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),
    LSTM(32),
    Dense(4)  # Predicting Open, Close, High, Low
])

model.compile(optimizer='adam', loss='mse')
model.summary()
```

```
# Step 7: Train model
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test,
y_test))

# Step 8: Plot training history
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.legend()
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()

# Step 9: Make predictions and inverse transform
predicted = model.predict(X_test)
predicted_prices = scaler.inverse_transform(predicted)
actual_prices = scaler.inverse_transform(y_test)

# Step 10: Plot predictions
plt.figure(figsize=(12,6))
plt.plot(actual_prices[:, 1], label='Actual Close')  # Close Price
plt.plot(predicted_prices[:, 1], label='Predicted Close')
plt.title("Google Stock Close Price Prediction (LSTM)")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Step-by-Step Summary:

1. **Install and Import Tools:**

   - We install `yfinance` to download stock data.

   - We import necessary Python libraries for data handling, scaling, modeling, and plotting.

2. **Get Google Stock Data:**

- We download historical stock data for Google (GOOG) from 2020 to 2024.

- We only keep the columns: `**Open`, `Close`, `High`, and `Low`.**

3. **Normalize Data:**

- Stock prices are scaled between 0 and 1 using `MinMaxScaler` to help the model train better.

4. **Create Sequences:**

- We split the data into small sequences (chunks) of 60 days each.

- The model learns from these 60-day chunks to predict the next day's prices.

5. **Split Data:**

- 80% of the sequences are used for training.

- 20% are used for testing (to see how well the model learned).

6. **Build LSTM Model:**

- We create a neural network with two LSTM layers and one final output layer.

- This model is trained to predict 4 values: Open, Close, High, and Low.

7. **Train the Model:**

- The model learns patterns in the data over 20 training rounds (called epochs).

8. **Plot Training Progress:**

- We draw a graph to see how the model's error reduces over time.

9. **Make Predictions:**

   - We test the model by letting it predict stock prices on new unseen data.

   - We convert predictions back to real price values.

10. **Show Results:**

   - We plot a graph comparing actual vs. predicted `Close` prices.

—

```
# Install required packages
!pip install --quiet tensorflow pillow

import numpy as np
import tensorflow as tf
from tensorflow.keras.applications.inception_v3 import InceptionV3, preprocess_input
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import get_file

# Load a sample image (inbuilt Keras image)
sample_image_path = tf.keras.utils.get_file('elephant.jpg',

'https://storage.googleapis.com/download.tensorflow.org/example_images/elephant.jpg')
```

```
# Preprocess image
def preprocess_image(image_path):
    image = load_img(image_path, target_size=(299, 299))
    image = img_to_array(image)
    image = np.expand_dims(image, axis=0)
    image = preprocess_input(image)
    return image

# Load InceptionV3 and extract features
def extract_features(image_path):
    model = InceptionV3(weights='imagenet')
    model_new = Model(model.input, model.layers[-2].output)  # Use second last layer for features
    image = preprocess_image(image_path)
    features = model_new.predict(image)
    return features

# Dummy tokenizer and word index (in real case, this comes from training data)
tokenizer = tf.keras.preprocessing.text.Tokenizer()
tokenizer.word_index = {'<start>': 1, 'a': 2, 'man': 3, 'riding': 4, 'an': 5, 'elephant': 6, '<end>': 7}
index_word = {v: k for k, v in tokenizer.word_index.items()}
vocab_size = len(tokenizer.word_index) + 1
max_length = 5
```

```
# STEP 1: Install and Import Libraries

!pip install -q keras


import numpy as np

import tensorflow as tf

from tensorflow.keras.applications.inception_v3 import InceptionV3,
preprocess_input

from tensorflow.keras.preprocessing.image import load_img, img_to_array

from tensorflow.keras.models import Model

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.layers import Input, LSTM, Embedding, Dense,
Dropout, add

import matplotlib.pyplot as plt

import requests

from PIL import Image

from io import BytesIO


# STEP 2: Load Sample Image (with User-Agent fix)

img_url =
"https://upload.wikimedia.org/wikipedia/commons/thumb/1/14/Gatto_europe
o4.jpg/250px-Gatto_europeo4.jpg"

headers = {'User-Agent': 'Mozilla/5.0 (compatible;
ImageCaptioningBot/1.0)'}

response = requests.get(img_url, headers=headers, stream=True)

response.raise_for_status()
```

```
img = Image.open(BytesIO(response.content)).resize((299, 299))

plt.imshow(img)

plt.axis('off')

plt.title("Sample Image")

plt.show()


# STEP 3: Preprocess Image for InceptionV3

def preprocess_image(img):

    x = img_to_array(img)

    x = np.expand_dims(x, axis=0)

    x = preprocess_input(x)

    return x


# STEP 4: Extract Features using InceptionV3

model_incep = InceptionV3(weights='imagenet')

model_incep = Model(model_incep.input, model_incep.layers[-2].output)
# use output before final softmax

image_features = model_incep.predict(preprocess_image(img))


# STEP 5: Dummy Tokenizer and Vocabulary

tokenizer = tf.keras.preprocessing.text.Tokenizer()

tokenizer.word_index = {'<start>':1, 'a':2, 'man':3, 'riding':4,
'horse':5, '<end>':6}

index_word = {v: k for k, v in tokenizer.word_index.items()}

vocab_size = len(tokenizer.word_index) + 1

max_length = 5
```

```python
# STEP 6: Define Captioning Model
def define_model(vocab_size, max_length):
    inputs1 = Input(shape=(2048,))
    fe1 = Dropout(0.5)(inputs1)
    fe2 = Dense(256, activation='relu')(fe1)

    inputs2 = Input(shape=(max_length,))
    se1 = Embedding(vocab_size, 256, mask_zero=True)(inputs2)
    se2 = Dropout(0.5)(se1)
    se3 = LSTM(256)(se2)

    decoder1 = add([fe2, se3])
    decoder2 = Dense(256, activation='relu')(decoder1)
    outputs = Dense(vocab_size, activation='softmax')(decoder2)

    model = Model(inputs=[inputs1, inputs2], outputs=outputs)
    return model

model = define_model(vocab_size, max_length)

# STEP 7: Dummy Inference Function
def generate_caption(model, tokenizer, photo, max_length=5):
    in_text = '<start>'
    for _ in range(max_length):
```

```
        sequence = tokenizer.texts_to_sequences([in_text])[0]

        sequence = pad_sequences([sequence], maxlen=max_length)

        yhat = model.predict([photo, sequence], verbose=0)

        yhat = np.argmax(yhat)

        word = index_word.get(yhat, '')

        if word == '<end>':

            break

        in_text += ' ' + word

    return in_text.replace('<start>', '').strip()


# STEP 8: Dummy Output (since weights are not trained)

print("\nPredicted Caption (dummy):")

print("a man riding horse")  # hardcoded output
```

8. Write a program to develop Autoencoders using MNIST Handwriten Digits

```
# STEP 1: Install and Import Libraries

import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras.datasets import mnist

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense, Flatten, Reshape


# STEP 2: Load MNIST Data

(x_train, _), (x_test, _) = mnist.load_data()

x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


# Flatten the images (28x28 -> 784)

x_train = x_train.reshape((len(x_train), 28 * 28))

x_test = x_test.reshape((len(x_test), 28 * 28))


# STEP 3: Define Autoencoder Model

# Encoding part

input_img = Input(shape=(784,))

encoded = Dense(128, activation='relu')(input_img)

encoded = Dense(64, activation='relu')(encoded)

encoded = Dense(32, activation='relu')(encoded)  # bottleneck
```

```
# Decoding part
decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(128, activation='relu')(decoded)
decoded = Dense(784, activation='sigmoid')(decoded)

# Autoencoder
autoencoder = Model(input_img, decoded)

# STEP 4: Compile Model
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# STEP 5: Train the Autoencoder
autoencoder.fit(
    x_train, x_train,
    epochs=10,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test, x_test)
)

# STEP 6: Encode and Decode Test Images
decoded_imgs = autoencoder.predict(x_test)

# STEP 7: Display Original and Reconstructed Images
```

```
n = 10
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis("off")

    # Reconstructed
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis("off")
plt.show()
```

✅ What this code does:

Loads MNIST digits (0–9)

Builds a simple autoencoder (encoder + decoder)

Trains the model

Reconstructs and displays original vs decoded image

6. Applying the Autoencoder algorithms for encoding real-world data

```
# Step 1: Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# Step 2: Load and preprocess the data
data = load_breast_cancer()
X = data.data
X = StandardScaler().fit_transform(X)  # Normalize

# Step 3: Split data
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

# Step 4: Build the Autoencoder
input_dim = X.shape[1]  # 30 features
```

```
encoding_dim = 10       # compressed to 10 features

input_layer = Input(shape=(input_dim,))
encoded = Dense(encoding_dim, activation='relu')(input_layer)
decoded = Dense(input_dim, activation='sigmoid')(encoded)

autoencoder = Model(inputs=input_layer, outputs=decoded)

# Step 5: Compile the model
autoencoder.compile(optimizer='adam', loss='mse')

# Step 6: Train the model
history = autoencoder.fit(X_train, X_train,
                          epochs=50,
                          batch_size=16,
                          shuffle=True,
                          validation_data=(X_test, X_test),
                          verbose=0)

# Step 7: Encode and Decode the test data
encoded_data = autoencoder.predict(X_test)
decoded_data = autoencoder.predict(X_test)

# Step 8: Visualize reconstruction error
reconstruction_error = np.mean(np.square(X_test - decoded_data), axis=1)
```

```
plt.hist(reconstruction_error, bins=30, color='skyblue')

plt.title("Reconstruction Error")

plt.xlabel("Error")

plt.ylabel("Frequency")

plt.show()
```

Autoencoder for Anomaly Detection

This program uses an autoencoder to detect anomalies in the Breast Cancer Wisconsin dataset. Here's a breakdown of the code:

Step 1: Import Libraries

The program imports necessary libraries, including:

- `numpy` and `pandas` for data manipulation
- `matplotlib` for visualization
- `sklearn` for data preprocessing and splitting
- `tensorflow` and `keras` for building the autoencoder model

Step 2: Load and Preprocess Data

The program loads the Breast Cancer Wisconsin dataset and normalizes the features using `StandardScaler`.

Step 3: Split Data

The program splits the data into training and testing sets using `train_test_split`.

Step 4: Build Autoencoder Model

The program builds an autoencoder model with:

- An input layer with 30 features (number of features in the dataset)
- An encoded layer with 10 features (compressed representation)
- A decoded layer with 30 features (reconstructed input)

The autoencoder model is defined using the `Model` API from Keras.

Step 5: Compile Model

The program compiles the autoencoder model with:

- Adam optimizer
- Mean Squared Error (MSE) loss function

Step 6: Train Model

The program trains the autoencoder model on the training data with:

- 50 epochs

- Batch size of 16

- Shuffle enabled

- Validation data set to the test data

Step 7: Encode and Decode Test Data

The program uses the trained autoencoder model to encode and decode the test data.

Step 8: Visualize Reconstruction Error

The program calculates the reconstruction error for each sample in the test data and visualizes it using a histogram. The reconstruction error is calculated as the mean squared difference between the original input and the reconstructed output.

Purpose:

The purpose of this program is to detect anomalies in the Breast Cancer Wisconsin dataset using an autoencoder. The autoencoder is trained to reconstruct the input data, and the reconstruction error is used to identify anomalies. Samples with high reconstruction error are likely to be anomalies.

Use Cases:

This program can be used for anomaly detection in various applications, such as:

- Medical diagnosis: Identifying unusual patterns in medical images or patient data.

- Network intrusion detection: Identifying unusual network traffic patterns.

- Quality control: Identifying defects in products or manufacturing processes.

4 . Image Segmentation

```
# U-Net Image Segmentation on Synthetic Data (Google Colab)

# Step 1: Install dependencies

!pip install -q tensorflow matplotlib


# Step 2: Import libraries

import numpy as np

import tensorflow as tf

from tensorflow.keras import layers, models

import matplotlib.pyplot as plt


# Step 3: Generate synthetic image segmentation data

def generate_synthetic_data(num_samples=100, image_size=64):

    images = np.zeros((num_samples, image_size, image_size, 1), dtype=np.float32)

    masks = np.zeros((num_samples, image_size, image_size, 1), dtype=np.float32)


    for i in range(num_samples):

        radius = np.random.randint(5, 15)

        center_x = np.random.randint(radius, image_size - radius)

        center_y = np.random.randint(radius, image_size - radius)

        y, x = np.ogrid[:image_size, :image_size]
```

```
        mask = (x - center_x) ** 2 + (y - center_y) ** 2 <= radius ** 2
        images[i, ..., 0] = mask.astype(np.float32) + 0.1 * np.random.randn(image_size,
image_size)
        masks[i, ..., 0] = mask.astype(np.float32)

    return images, masks

# Generate synthetic data
X, y = generate_synthetic_data(200)
X_train, X_test = X[:160], X[160:]
y_train, y_test = y[:160], y[160:]

# Step 4: Define a simple U-Net model
def build_unet(input_shape=(64, 64, 1)):
    inputs = layers.Input(input_shape)

    # Encoder
    c1 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(inputs)
    c1 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D((2, 2))(c1)

    c2 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D((2, 2))(c2)
```

```
    # Bottleneck
    c3 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(p2)
    c3 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c3)

    # Decoder
    u1 = layers.UpSampling2D((2, 2))(c3)
    u1 = layers.Concatenate()([u1, c2])
    c4 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(u1)
    c4 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(c4)

    u2 = layers.UpSampling2D((2, 2))(c4)
    u2 = layers.Concatenate()([u2, c1])
    c5 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(u2)
    c5 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(c5)

    outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(c5)

    model = models.Model(inputs, outputs)
    return model

# Step 5: Compile and train the model
model = build_unet()
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=10, batch_size=16, validation_split=0.1)
```

```
# Step 6: Evaluate and visualize results
def visualize_predictions(images, masks, predictions, num=5):
    plt.figure(figsize=(12, num * 3))
    for i in range(num):
        plt.subplot(num, 3, i * 3 + 1)
        plt.title("Input")
        plt.imshow(images[i].squeeze(), cmap='gray')
        plt.axis('off')

        plt.subplot(num, 3, i * 3 + 2)
        plt.title("Ground Truth")
        plt.imshow(masks[i].squeeze(), cmap='gray')
        plt.axis('off')

        plt.subplot(num, 3, i * 3 + 3)
        plt.title("Prediction")
        plt.imshow(predictions[i].squeeze() > 0.5, cmap='gray')
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# Predict on test set
preds = model.predict(X_test)
visualize_predictions(X_test, y_test, preds)
```

What is Image Segmentation?

Imagine you have a photo and you want to **highlight only the important object** in it — like a person, a dog, or a car — and ignore the rest.

Image segmentation does exactly that. It **separates the object from the background** — like coloring only the dog in a photo and leaving everything else gray.

What Did the Code Do?

1. **Created Fake Images**
   The code made simple black and white images with **circles** in them. These circles are the "objects" we want to find.
2. **Built a Smart Model (U-Net)**
   We used a small brain-like program called a **U-Net model**. It learns to recognize where the circles are in the images.
3. **Trained the Model**
   We showed the model many fake images and their correct answers (where the circles are), so it can learn how to do it by itself.
4. **Tested the Model**
   After training, we gave the model new images it hadn't seen before and asked: "Can you find the circle here too?"
5. **Showed the Results**
   It showed us:
   - The input image (with a circle)
   - The correct answer (where the circle really is)
   - The model's guess (where it thinks the circle is)

What to Look For

- **Good Result**: The prediction and ground truth look almost the same.
- **Bad Result**: The prediction is missing the circle, or it's in the wrong place or shape.

```
# Step 1: Install required packages

!pip install -q tensorflow matplotlib
```

```
# Step 2: Import necessary libraries
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

# Step 3: Set parameters
img_shape = (28, 28, 1)  # Output image shape (28x28 grayscale images)
z_dim = 100  # Latent space dimension (random noise)

# Step 4: Define the Generator Model
def build_generator():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(128, activation='relu', input_dim=z_dim),
        tf.keras.layers.Dense(784, activation='sigmoid'),
        tf.keras.layers.Reshape(img_shape)
    ])
    return model

# Step 5: Define the Discriminator Model
def build_discriminator():
    model = tf.keras.Sequential([
        tf.keras.layers.Flatten(input_shape=img_shape),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])
```

```
    return model


# Step 6: Build the GAN model by combining the generator and discriminator
def build_gan(generator, discriminator):
    discriminator.trainable = False
    model = tf.keras.Sequential([generator, discriminator])
    return model


# Step 7: Compile the models
discriminator = build_discriminator()
discriminator.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])


generator = build_generator()
gan = build_gan(generator, discriminator)
gan.compile(optimizer='adam', loss='binary_crossentropy')


# Step 8: Load dataset (using MNIST for simplicity)
(X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
X_train = (X_train / 127.5) - 1.0  # Normalize images to [-1, 1]
X_train = np.expand_dims(X_train, axis=3)  # Add channel dimension


# Step 9: Train the GAN model
def train_gan(epochs, batch_size, save_interval):
    # Labels for real and fake images
    real_labels = np.ones((batch_size, 1))
```

```
    fake_labels = np.zeros((batch_size, 1))

    for epoch in range(epochs):
        # Select a random batch of real images
        idx = np.random.randint(0, X_train.shape[0], batch_size)
        real_images = X_train[idx]

        # Generate a batch of fake images
        noise = np.random.normal(0, 1, (batch_size, z_dim))
        fake_images = generator.predict(noise)

        # Train the discriminator
        d_loss_real = discriminator.train_on_batch(real_images, real_labels)
        d_loss_fake = discriminator.train_on_batch(fake_images, fake_labels)
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train the generator (via the GAN model)
        noise = np.random.normal(0, 1, (batch_size, z_dim))
        g_loss = gan.train_on_batch(noise, real_labels)

        # Save generated images at specified intervals
        if epoch % save_interval == 0:
            save_generated_images(epoch)

# Step 10: Function to save and display generated images with improvement
```

```
def save_generated_images(epoch, examples=5, dim=(1, 5), figsize=(10, 2)):
    noise = np.random.normal(0, 1, (examples, z_dim))
    generated_images = generator.predict(noise)
    generated_images = (generated_images + 1) / 2  # Rescale to [0, 1]

    # Display generated images
    plt.figure(figsize=figsize)
    for i in range(examples):
        plt.subplot(dim[0], dim[1], i + 1)
        plt.imshow(generated_images[i, :, :, 0], cmap='gray')
        plt.axis('off')
    plt.tight_layout()
    plt.suptitle(f'Generated Images at Epoch {epoch}', fontsize=16)
    plt.show()

# Step 11: Run the GAN training for 1000 epochs, saving every 100 epochs to show
improvement
epochs = 1000
batch_size = 64
save_interval = 100
train_gan(epochs, batch_size, save_interval)
```

What This Code Does:

1. **Generates Fake Images**: The generator tries to create fake images resembling handwritten digits.
2. **Improves Over Time**: The GAN is trained for 1000 epochs, and the **generator** improves its ability to create more realistic images as the discriminator gets better at distinguishing real from fake images.
3. **Displays Images**: Every **100 epochs**, it shows the current generated images. Initially, the images will look like random noise, but as the GAN trains, the generated images will gradually become more recognizable as **handwritten digits**.

Key Improvements:

- The **generated images** are displayed with titles indicating the **epoch number**, showing how the images improve over time.
- You will visually notice how the GAN's **generated images** improve as it trains: the generated digits will start as random noise and become more coherent as the model trains.

Example Output:

- At **Epoch 0**, the images will look like noise.
- At **Epoch 100**, the images will look a little more structured but still not perfect.
- By **Epoch 1000**, the images will closely resemble digits like **0, 1, 2, etc.**.

Additional Notes:

1. **GPU Utilization**: If you're running this on **Google Colab**, make sure to enable **GPU** acceleration in the runtime settings for faster training.
   - Go to `Runtime` > `Change runtime type` > Select `GPU`.
2. **Saving Images**: This code displays the images during training, but you can also save them to disk if needed by modifying the `save_generated_images` function.
3. **Customization**: You can adjust the number of **epochs**, **batch size**, and **save intervals** based on how frequently you want to see the progress.